CS336 Assignment 2: Systems and Parallelism
Writeup

Your Name
SUNet ID

Spring 2026

Contents

1 Profiling and Benchmarking **2**
1.1 End-to-End Benchmarking . . . 2
1.2 Nsight Systems Profiling . . . 5
1.3 Mixed Precision . . . 8
1.3.1 Mixed-Precision Accumulation . . . 8
1.3.2 Benchmarking Mixed Precision . . . 8
1.4 Memory Profiling . . . 9

2 Single-GPU Memory **13**

3 GPU Kernels **15**
3.1 PyTorch Attention Benchmarking . . . 15
3.2 Torch Compile . . . 16
3.3 FlashAttention-2 Benchmarking . . . 18

4 Distributed Data Parallel Training **18**
4.1 Distributed Communication . . . 18
4.2 Naïve DDP Benchmarking . . . 18
4.3 Minimal DDP with Flat Gradients . . . 18
4.4 DDP with Overlapping Individual Parameters . . . 18

5 Optimizer State Sharding **18**

6 Fully-Sharded Data Parallel **19**

7 Analyzing Parallelism Strategies **19**
7.1 Communication Primitives . . . 19
7.2 Data Parallel Calculations . . . 19
7.3 FSDP Calculations . . . 19
7.4 Tensor Parallel Calculations . . . 19
7.5 2D Parallelism (FSDP + TP) . . . 20

8 Leaderboard **20**

1 Profiling and Benchmarking

1.1 End-to-End Benchmarking

Problem (benchmarking_script)

(b) Timings for forward, backward, and optimizer step. Table 1 summarizes the mean timings across four model sizes (5 warm-up steps, 10 measurement iterations each, batch size 4, context length 512). The 10B model (12.8B params) OOM'd on a single 96 GB GPU even at batch size 1.

As model size grows from small (129M) to xl (3.4B), forward time scales from 8.0ms to 158ms ($\sim$20$\times$), backward from 24ms to 605ms ($\sim$25$\times$), and optimizer from 28ms to 364ms ($\sim$13$\times$). The backward-to-forward ratio ranges from $\sim$3$\times$ (small) to $\sim$3.8$\times$ (xl), higher than the theoretical 2$\times$ due to the memory-bound nature of non-matmul backward kernels at these model scales.

Table 1: Mean benchmark timings across model sizes (5 warmup, 10 measurement steps, seconds).

Size	Params	Forward	Backward	Optimizer	Total
small	128.6M	0.0080	0.0240	0.0278	0.0645
medium	423.2M	0.0163	0.1005	0.0491	0.1727
large	969.4M	0.0606	0.2242	0.0883	0.3824
xl	3406.8M	0.1577	0.6049	0.3645	1.1884
10B	12832.8M	OOM (single GPU, 96 GB)			

Detailed per-iteration timings for each model size are reported in Tables 2–5.

Table 2: Per-iteration timings — small (128.6M params, seconds).

Iter	Forward	Backward	Optimizer	Total
0	0.008805	0.027929	0.027512	0.068860
1	0.007973	0.023553	0.027840	0.063957
2	0.007969	0.023524	0.027921	0.064068
3	0.007997	0.023520	0.027875	0.064041
4	0.007806	0.023695	0.027844	0.063934
5	0.007815	0.023712	0.027913	0.064079
6	0.007880	0.023598	0.027878	0.063962
7	0.007954	0.023538	0.027915	0.064049
8	0.007936	0.023600	0.027847	0.064024
9	0.008003	0.023536	0.027891	0.064026

Table 3: Per-iteration timings — medium (423.2M params, seconds).

Iter	Forward	Backward	Optimizer	Total
0	0.022665	0.103150	0.049132	0.181706
1	0.015675	0.100775	0.049791	0.172990
2	0.015603	0.101244	0.049703	0.173358
3	0.015699	0.100482	0.050023	0.173115
4	0.015801	0.100800	0.049977	0.173389
5	0.015657	0.100513	0.048556	0.171523
6	0.015449	0.098595	0.048483	0.169038
7	0.015732	0.100194	0.048560	0.171374
8	0.015141	0.098973	0.048456	0.168943
9	0.015582	0.100286	0.048559	0.171229

Table 4: Per-iteration timings — large (969.4M params, seconds).

Iter	Forward	Backward	Optimizer	Total
0	0.069621	0.225146	0.087279	0.391637
1	0.059705	0.224016	0.088500	0.381821
2	0.059492	0.224260	0.088216	0.381226
3	0.059803	0.223808	0.088319	0.381162
4	0.059690	0.224178	0.088502	0.381656
5	0.059637	0.223993	0.088518	0.381416
6	0.059546	0.223956	0.088468	0.381155
7	0.059490	0.224351	0.088252	0.381292
8	0.059381	0.224349	0.088170	0.381223
9	0.059410	0.224161	0.088399	0.381158

Table 5: Per-iteration timings — xl (3.41B params, seconds).

Iter	Forward	Backward	Optimizer	Total
0	0.213040	0.609433	0.361567	1.244934
1	0.151633	0.603544	0.364648	1.181239
2	0.151771	0.604308	0.365424	1.182898
3	0.151549	0.604199	0.365209	1.182638
4	0.151668	0.604337	0.364411	1.181654
5	0.151368	0.605109	0.365054	1.182857
6	0.151595	0.603789	0.364284	1.181087
7	0.151271	0.605567	0.364650	1.182793
8	0.151552	0.604438	0.364929	1.182570
9	0.151519	0.604381	0.364340	1.181539

(c) Effect of removing warm-up steps. We re-run the large model (969M params) with warmup $\in \{0, 1, 5\}$ to isolate the effect of warm-up on measured timings (Table 6).

Without warm-up (warmup=0), the first iteration is dramatically slower: the forward pass takes 295ms compared to ∼62ms in steady state (a ∼4.8× slowdown), and the backward pass reaches 286ms vs. ∼224ms. The total for iteration 0 is 629ms—1.6× the steady-state ∼384ms. This is because CUDA performs lazy initialization on the first kernel launch, including context creation, kernel JIT compilation, and memory pool allocation.

With warmup=1, the first measured iteration still shows overhead (73ms forward vs. ∼60ms steady state), because a single warm-up step only partially amortizes cuDNN autotuning and memory allocator warm-up. By warmup=5 (our default), the overhead on iteration 0 shrinks to 69ms forward—only ∼16% above steady state—and iterations 1–9 are fully stable at ∼381ms total.

The steady-state timings (iterations 4–9) are consistent across all warm-up settings at ∼381–385ms total, confirming that warm-up only affects early measurements and 5 warm-up steps are sufficient for reliable benchmarking.

Table 6: Warm-up effect on the large model (969M params, seconds).

(a) warmup = 0

Iter	Forward	Backward	Optimizer	Total
0	0.294910	0.285918	0.039139	0.629414
1	0.063381	0.236767	0.091641	0.401459
2	0.063882	0.228353	0.090433	0.392371
3	0.063467	0.226980	0.090378	0.390437
4	0.062608	0.224941	0.088858	0.386012
5	0.062538	0.224086	0.088961	0.385279
6	0.061851	0.223798	0.088168	0.383509
7	0.061446	0.224394	0.088809	0.384442
8	0.061604	0.223993	0.088839	0.384135
9	0.062133	0.224124	0.089207	0.385142

(b) warmup = 1

Iter	Forward	Backward	Optimizer	Total
0	0.072542	0.225798	0.087844	0.395896
1	0.061591	0.223565	0.088606	0.383297
2	0.061285	0.223906	0.088446	0.383004
3	0.062142	0.223773	0.088503	0.383811
4	0.059694	0.224057	0.088454	0.381622
5	0.059697	0.224042	0.088680	0.382038
6	0.059696	0.223751	0.088694	0.381424
7	0.059165	0.224367	0.088501	0.381191
8	0.059721	0.223903	0.088787	0.381676
9	0.059531	0.224207	0.088459	0.381434

(c) warmup = 5 (default)

Iter	Forward	Backward	Optimizer	Total
0	0.069072	0.225348	0.087045	0.390757
1	0.059510	0.224080	0.088312	0.381124
2	0.059796	0.223846	0.088253	0.381137
3	0.060320	0.223399	0.088299	0.381211
4	0.059558	0.224158	0.088539	0.381545
5	0.059449	0.224155	0.088413	0.381281
6	0.059837	0.224020	0.088368	0.381433
7	0.059941	0.223970	0.088252	0.381344
8	0.059600	0.224101	0.088168	0.381192
9	0.059407	0.224022	0.088702	0.381272

1.2 Nsight Systems Profiling

Problem (nsys_profile)

We profiled two model sizes—**small** (128.6M params) and **large** (969.4M params)—at three

power-of-two context lengths each, using `nsys profile` with 5 warmup + 5 measurement iterations. The small model was profiled at context lengths 256, 1024, and 2048 (the maximum that fits in GPU memory); the large model at 256, 512, and 1024.

(a) Total time on forward pass. Table 7 shows the GPU-projected forward pass time per iteration from the `nvtx_gpu_proj_sum` report. The GPU-projected time measures the actual GPU execution span of kernels launched within the `forward_pass` NVTX range, and is generally *higher* than the Python `timeit` measurement (which only records CPU dispatch time without `torch.cuda.synchronize()`). For example, for the large model at context length 512, nsys reports a GPU forward time of 111.2 ms, while `timeit` records ∼60.6 ms—reflecting only how long the CPU takes to dispatch forward kernels asynchronously. The wall-clock total per iteration (∼382 ms with synchronization) is consistent with the sum of GPU forward, backward, and optimizer times.

Table 7: GPU-projected forward pass time per iteration (ms).

Model	Context	GPU Forward (ms)	Python `timeit` Forward (ms)
small	256	11.9	—
small	1024	50.9	—
small	2048	145.3	—
large	256	60.0	—
large	512	111.2	60.6 (ctx=512 benchmark)
large	1024	300.4	—

The forward pass time grows roughly quadratically with context length (e.g., small: 11.9 → 50.9 → 145.3 ms for 256 → 1024 → 2048), consistent with the $O(T^2)$ attention scaling dominating at longer sequences.

(b) Most time-consuming CUDA kernel. The most time-consuming CUDA kernel across all profiles is the CUTLASS SGEMM kernel (single-precision general matrix multiply), which implements all matmul operations in the Transformer (linear projections, attention score computation, FFN layers). Table 8 shows the top kernel for each configuration.

Table 8: Top CUDA kernel by cumulative GPU time (full training step).

Model	Context	Top Kernel	Time %	Invocations
small	256	`sgemm_128x64_nn`	16.9%	840
small	1024	`sgemm_128x64_nn`	13.3%	840
small	2048	`sgemm_128x64_nn`	10.2%	840
large	256	`sgemm_128x256_tn`	24.3%	2530
large	512	`sgemm_128x256_tn`	23.3%	3240
large	1024	`sgemm_128x256_tn`	16.9%	2530

The same GEMM kernel class dominates both the forward-only and forward+backward profiles. During a full training step, GEMM kernels remain the most time-consuming category, though their collective share decreases relative to forward-only because the backward pass and optimizer introduce additional elementwise and reduction operations.

(c) Non-matmul kernels with non-trivial runtime. Table 9 shows that GEMM kernels account for 49–61% of total GPU kernel time during a full training step. The remaining time is spent on several non-matmul kernel categories:

- `vectorized_elementwise_kernel` (mul, add): 6–9% each, implementing LayerNorm scaling, residual additions, and activation functions.
- `elementwise_kernel` (div, subtract): 3–7%, used in softmax normalization and gradient computation.
- `reduce_kernel`: ∼3%, performing reductions for LayerNorm mean/variance and loss computation.

Table 9: GEMM kernel fraction of total GPU kernel time (full training step).

Model	Context	GEMM %	Non-GEMM %
small	256	58.8	41.2
small	1024	43.4	56.6
small	2048	32.1	67.9
large	256	61.3	38.7
large	512	60.5	39.5
large	1024	49.1	50.9

(d) Full training step profile comparison. Comparing the GEMM fraction across configurations (Table 9), we observe two trends: (1) the GEMM fraction decreases with increasing context length (e.g., small: 58.8% → 43.4% → 32.1% for 256 → 1024 → 2048), because attention-related elementwise operations (softmax, masking, scaling) grow as $O(T^2)$ while FFN matmuls grow only as $O(T)$; and (2) during a full training step (forward + backward + optimizer), the GEMM fraction is lower than inference-only would suggest because the backward pass adds gradient elementwise operations and the AdamW optimizer performs per-parameter elementwise updates (momentum, variance, weight decay) that are entirely non-GEMM.

(e) Softmax vs. matrix multiplication runtime comparison. Table 10 shows the GPU-projected time per call (median) for the three annotated sub-operations within `scaled_dot_product_attention`: the QK^T matmul (`computing attention scores`), softmax, and the attn × V matmul (`final matmul`).

Table 10: Attention sub-operation GPU time per call (median, μs).

Model	Context	QK^T (μs)	Softmax (μs)	Attn×V (μs)	Softmax / Matmul
small	256	23.8	41.5	23.5	0.88×
small	1024	465.7	1115.9	226.1	1.61×
small	2048	1699.7	4413.5	698.1	1.84×
large	256	29.5	58.5	25.3	1.07×
large	512	119.6	271.2	88.8	1.30×
large	1024	766.1	1863.2	320.4	1.72×

Despite having ∼25× fewer FLOPs than the two attention matmuls combined ($O(T^2)$ for softmax vs. $O(T^2 \cdot d_{\text{head}})$ for each matmul, where $d_{\text{head}} = 64$), softmax takes *comparable or even*

more wall-clock time. This is because softmax is **memory-bound**: it must read and write the entire $[B, H, T, T]$ attention score matrix multiple times (for row-max subtraction, exponentiation, row-sum, and division), while the matmul kernels are **compute-bound** and can effectively utilize the GPU's SIMT units at high arithmetic intensity. As context length increases, the softmax-to-matmul runtime ratio grows (from $\sim$0.9$\times$ to $\sim$1.8$\times$), because the memory-bound softmax becomes increasingly dominant over the compute-bound matmuls at larger attention matrices.

1.3 Mixed Precision

1.3.1 Mixed-Precision Accumulation

Problem (mixed_precision_accumulation)

Listing 1: Mixed-Precision Accumulation

```
import torch

s = torch.tensor(0, dtype=torch.float32)
for i in range(1000):
  s += torch.tensor(0.01, dtype=torch.float32)
print(s) # tensor(10.0001)

s = torch.tensor(0, dtype=torch.float16)
for i in range(1000):
  s += torch.tensor(0.01, dtype=torch.float16)
print(s) # tensor(9.9531, dtype=torch.float16)

s = torch.tensor(0, dtype=torch.float32)
for i in range(1000):
  s += torch.tensor(0.01, dtype=torch.float16)
print(s) # tensor(10.0021)

s = torch.tensor(0, dtype=torch.float16)
for i in range(1000):
  s += torch.tensor(0.01, dtype=torch.float32)
print(s) # tensor(9.9531, dtype=torch.float16)
```

1.3.2 Benchmarking Mixed Precision

Problem (benchmarking_mixed_precision)

(a) Data types under FP16 autocasting.

- the model parameters within the autocast context?: **FP32**
- the output of the first feed-forward layer? **FP16**
- the output of layer norm? **FP32**, since layer norm has reduce operation (mean)
- the model's predicted logits? **FP16**, from the last fc2, which will use FB16
- the loss? **FP32**
- the model's gradients? **FP32**

(b) Layer normalization and BF16. For FP16 mixed precision, layer norm's mean/variance computation and the scaling by β and α are numerically sensitive, because they involve reductions and divisions that can loss precision and underflow in FP16.

(c) Mixed precision benchmarking results. Table 11 compares median per-step timings (seconds) for FP32 vs. BF16 mixed precision across all model sizes (batch 4, context 512, 10 iterations after 5 warmup steps). The 10B model OOMs in both precisions.

Table 11: FP32 vs. BF16 median per-phase and total step time (seconds). Each phase is timed with `torch.cuda.synchronize()` barriers.

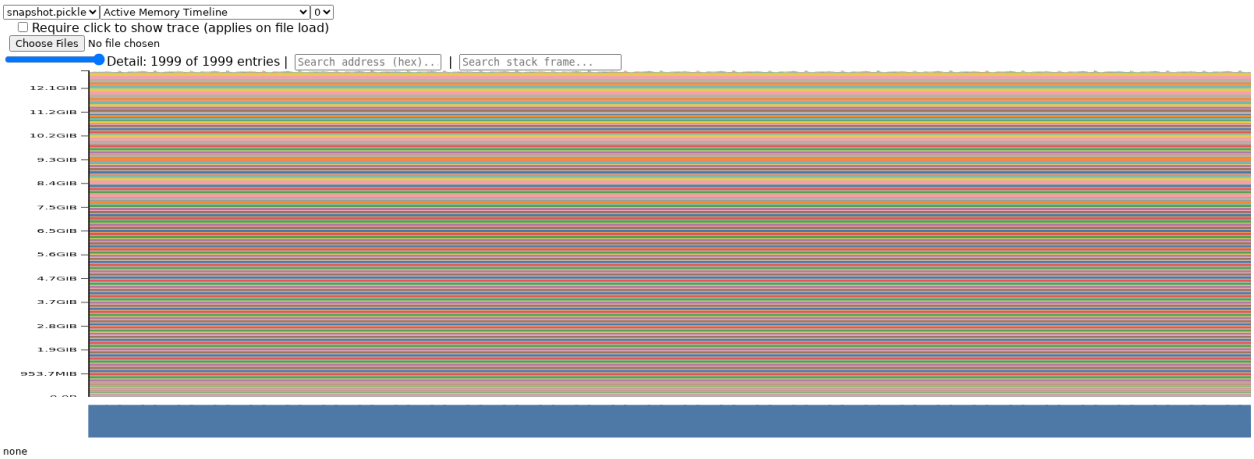
		Forward		Backward		Optimizer		
Size	Dtype	Time	Sp.	Time	Sp.	Time	Sp.	Total Sp.
Small	FP32	0.0188		0.0371		0.0086		—
(0.13B)	BF16	0.0093	2.0×	0.0203	1.8×	0.0081	1.1×	1.71×
Medium	FP32	0.0497		0.0987		0.0225		—
(0.42B)	BF16	0.0215	2.3×	0.0525	1.9×	0.0222	1.0×	1.78×
Large	FP32	0.1111		0.2236		0.0479		—
(0.97B)	BF16	0.0417	2.7×	0.1091	2.1×	0.0481	1.0×	1.93×
XL	FP32	0.3255		0.5802		0.2779		—
(3.4B)	BF16	0.1009	3.2×	0.2387	2.4×	0.2776	1.0×	1.92×

BF16 mixed precision yields a 1.71×–1.93× end-to-end speedup. The per-phase breakdown (with `torch.cuda.synchronize()` barriers between phases) reveals a clear pattern: the **forward pass** benefits most (2.0–3.2×), with speedup growing as model size increases and GEMMs become more compute-bound; the **backward pass** also accelerates (1.8–2.4×) because autograd reuses the BF16 activations saved during the forward pass; the **optimizer step** is essentially unchanged ($\approx 1.0\times$) because AdamW operates on FP32 master weights regardless of autocast. The forward-pass speedup exceeds 2× even for the smallest model, confirming that BF16 Tensor Cores provide genuine throughput gains rather than just memory savings.

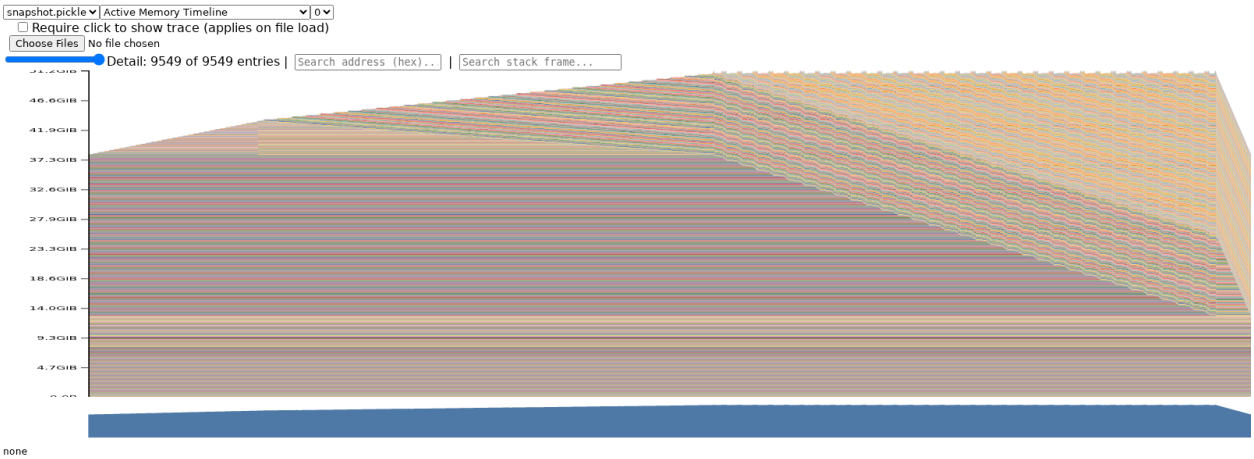
1.4 Memory Profiling

Problem (memory_profiling)

(a) Memory timelines. We profiled the xl model (3.4B parameters) at context length 128 using `torch.cuda.memory._record_memory_history` and visualised the snapshots with PyTorch's `memory_viz` tool.



(a) Inference only (forward pass). Memory is dominated by the 12.7 GiB of model weights; transient activations produce thin spikes.



(b) Full training step. The forward pass builds up saved-for-backward activations (rising slope), the backward pass frees them while allocating gradients (peak then decline), and the optimizer step keeps memory flat.

Figure 1: Active memory timeline for the xl model (ctx=128) from PyTorch `memory_viz`.

Inference-only (forward pass): The timeline shows a single "staircase" pattern — memory

climbs layer-by-layer as each `TransformerBlock` computes its output, then stays flat after the final layer. No gradients or optimizer states are allocated, so the peak (≈12.9 GiB) sits only slightly above the model-weight baseline (≈12.7 GiB). Intermediate tensors are allocated and freed within each layer, producing small sawtooth spikes on top of the staircase.

Full training step (forward + backward + optimizer): The timeline has three distinct phases: (1) the forward pass creates the same staircase, but now all activations saved for autograd ("residuals") remain allocated, pushing memory steadily upward; (2) the backward pass then frees those residuals layer-by-layer *while* allocating gradient tensors, producing a characteristic "mountain" that peaks at the transition point (≈51.4 GiB overall, since optimizer states are already present from warmup); (3) the optimizer step performs in-place updates, so memory stays roughly flat. The three phases are clearly distinguishable from the shape of the timeline.

(See `benchmark_results/memory/xl_ctx128_inference.html` and `xl_ctx128_train.html` for the full interactive timelines.)

Table 12: Peak GPU memory allocated (GiB) for the xl model at different context lengths.

Context length	Inference only	Full training
128 (batch 4)	12.93	51.42
2048 (batch 4)	21.60	OOM (>95 GiB)
2048 (batch 1)	—	91.16

(b) Peak memory usage. At context length 128 the forward-only peak is dominated by the model weights (4×3.4B $= 12.7$ GiB in FP32); the additional activation memory is negligible. Training adds optimizer states ($2 \times 12.7 = 25.4$ GiB for AdamW first and second moments), gradients (12.7 GiB), and saved-for-backward activations, totalling 51.4 GiB.

At context length 2048, inference peak grows to 21.6 GiB because attention score matrices scale as $O(\text{seq}^2)$; training at batch 4 exceeds the 95 GiB GPU capacity and OOMs. Reducing to batch 1 allows training to complete at 91.2 GiB.

Table 13: Peak GPU memory (GiB) for the xl model: FP32 vs. BF16 autocast.

	Inference only		Full training	
Context (batch)	FP32	BF16	FP32	BF16
128 (batch 4)	12.93	19.15	51.42	51.41
2048 (batch 4)	21.60	25.50	OOM	OOM
2048 (batch 1)	—	—	91.16	82.49

(c) Mixed precision effect on memory. The effect of BF16 mixed precision on memory depends on the balance between two opposing factors: `torch.autocast` keeps model parameters in FP32 and lazily caches BF16 copies of each weight matrix (≈6.3 GiB overhead), but activations and saved-for-backward tensors are stored in BF16 (half the FP32 size). For **inference at short context** (128), the weight-cache overhead dominates and BF16 uses *more* memory (+6.2 GiB). For **training at short context**, the two effects nearly cancel: 51.42 vs. 51.41 GiB. For **training at long context** (2048, batch 1), the larger activation volume tips the balance and BF16 *saves*

8.7 GiB (91.16 → 82.49 GiB). In summary, mixed precision does not significantly reduce memory for short-context workloads but provides meaningful savings when activation memory dominates (long context, large batch).

(d) Size of activation tensor in residual stream. For the xl model with batch size B=4, context length T=512, and d_{model}=2560, the residual-stream activation tensor has shape $(B, T, d_{\text{model}}) = (4, 512, 2560)$. In FP32, each element is 4 bytes, so the tensor size is:

$$4 \times 512 \times 2560 \times 4 \text{ bytes} = 20\,971\,520 \text{ bytes} = \frac{20\,971\,520}{1024^2} = 20 \text{ MiB}.$$

(e) Largest allocations in memory timeline. When reducing the "Detail" level in `pytorch.org/memory_viz`, the two dominant allocation sizes that remain visible are:

- **100 MiB** – FFN weight matrices. Each has shape $d_{\text{model}} \times d_{\text{ff}} = 2560 \times 10240$ in FP32: $2560 \times 10240 \times 4/1024^2 = 100$ MiB. There are 96 such blocks (32 layers × 3 FFN weights per layer: gate, up, and down projections), totalling 9.38 GiB.
- **∼25 MiB** – Attention projection weight matrices. Each has shape $d_{\text{model}} \times d_{\text{model}} = 2560 \times 2560$ in FP32: $2560 \times 2560 \times 4/1024^2 = 25$ MiB (allocated as 26 MiB by the CUDA caching allocator). There are 128 such blocks (32 layers × 4 projections: W_Q, W_K, W_V, W_O), totalling 3.25 GiB.

Together these model-parameter allocations account for $9.38 + 3.25 = 12.63$ GiB, consistent with the ∼12.7 GiB baseline observed in the memory timeline. The stack traces confirm they come from `torch.nn.Parameter` initialization inside `TransformerBlock` linear layers.

(f) Memory saved for backward per TransformerBlock. Using `nsys profile --cuda-memory-usage=tru -t cuda,nvtx` combined with per-block NVTX ranges and `torch.cuda.memory_allocated()` hooks, we measured memory allocated during a single `TransformerBlock` forward pass on the xl model (batch 4, ctx 128, FP32). Each block saves **∼166 MiB** of activations for the backward pass.

Table 14: Per-operation breakdown of saved-for-backward memory within one `TransformerBlock` (xl, ctx=128, FP32).

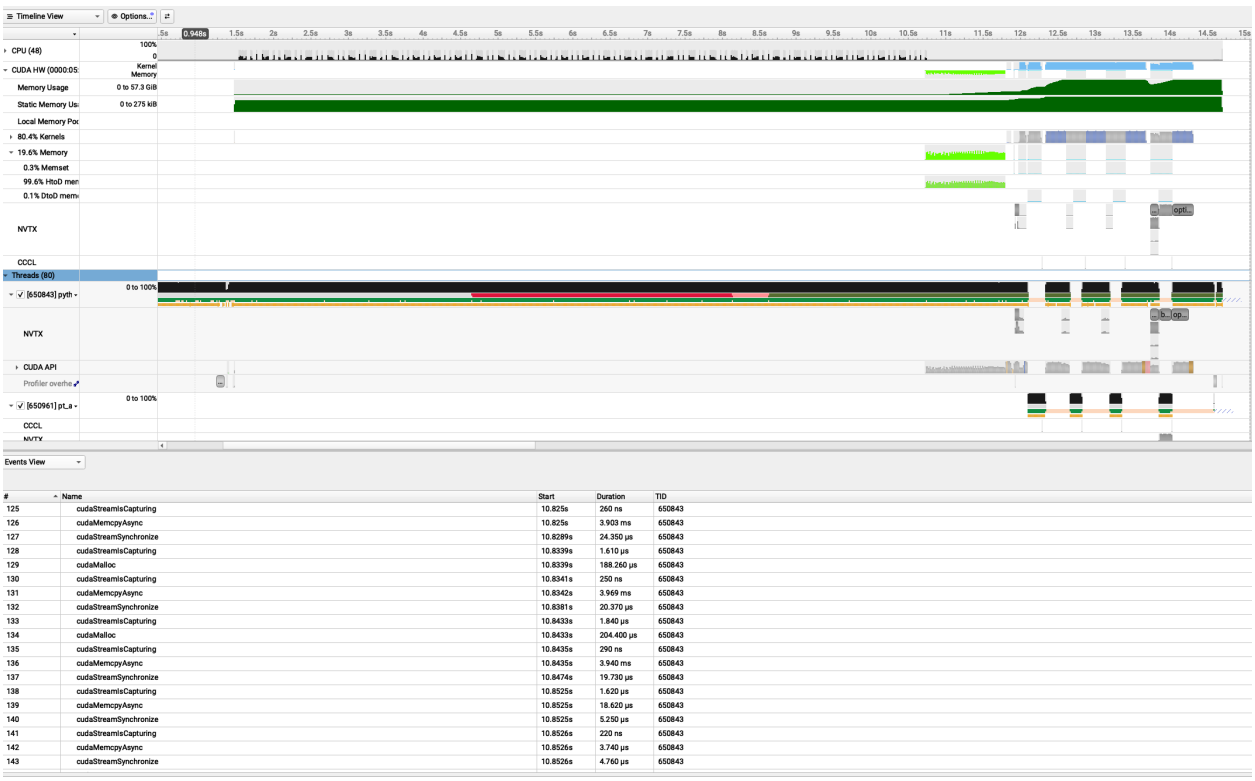
Operation	Δ Memory (MiB)	% of Total
FFN (SwiGLU)	105.0	63.2%
Attention (MHSA)	41.2	24.8%
RMSNorm (ln1)	10.0	6.0%
RMSNorm (ln2)	10.0	6.0%
Residual add (net)	0.0	0.0%
Total per block	166.2	100%

SwiGLU dominates because its three linear projections (w_1, w_2, w_3) each save their input tensors for `backward`, and the element-wise `silu` and multiplication further save intermediate activations (five d_{ff}-sized tensors of 20 MiB each + one d_{model}-sized input of 5 MiB = 105 MiB).

Backward-pass gradient memory. During backward through each block, the saved-for-backward tensors (∼166 MiB) are freed, but new gradient tensors are created. We observed:

- Forward delta (saved for backward): +166.3 MiB per block.
- Backward net memory change: +238.7 MiB per block.
- Gradient memory created = freed + net change = $166.3 + 238.7 = 405.0$ MiB.

Each `TransformerBlock` has 104,862,720 weight parameters, so the expected gradient memory is $104{,}862{,}720 \times 4$ bytes = 400.0 MiB. The observed 405 MiB is within 1.3% of the expected value; the small overhead comes from temporary intermediate gradient tensors allocated during backpropagation through composite operations. This confirms the result matches expectation.



(a) nsys memory view.

2 Single-GPU Memory

Problem (gradient_checkpointing)

(a) Optimal checkpointing strategy. The strategy that minimises peak activation memory (ignoring compute) is **recursive binary checkpointing**. Split the N Transformer blocks into two halves; wrap the first half in a `checkpoint` call (which discards its internal residuals and saves only the input tensor), then recursively apply the same split to both halves. The base case is a single block, which runs normally.

Concrete example ($N = 8$).

```
Level 1: checkpoint([0-3]),  recurse [4-7]
  Level 2: checkpoint([4-5]),  recurse [6-7]
    Level 3: checkpoint([6]),  run block 7
```

During backward, PyTorch recomputes each checkpointed segment on demand. At any instant the memory holds:

- **1 block's full residuals** (the block currently being differentiated, ∼166 MiB for xl);
- $\log_2 N$ **checkpoint boundary tensors** (one saved input per recursion level, each ∼5 MiB for xl).

For $N = 8$: peak $\approx 166 + 3 \times 5 = 181$ MiB instead of $8 \times 166 = 1328$ MiB without checkpointing. For $N = 32$: peak $\approx 166 + 5 \times 5 = 191$ MiB instead of $32 \times 166 = 5312$ MiB.

Asymptotic analysis. Peak activation memory is $O(\log N)$ (one residual-stream-sized tensor per recursion level, plus one block's residuals which is $O(1)$). Compute cost is $O(N \log N)$ forward-pass equivalents, since each block is recomputed $O(\log N)$ times across recursion levels.

Code sketch.

```
from torch.utils.checkpoint import checkpoint

def recursive_forward(blocks, x):
    if len(blocks) == 1:
        return blocks[0](x)          # base case
    mid = len(blocks) // 2
    # First half: discard residuals, save only input x
    x = checkpoint(recursive_forward, blocks[:mid], x,
                   use_reentrant=False)
    # Second half: recurse (same strategy)
    return recursive_forward(blocks[mid:], x)

# Usage inside the model's forward():
#   x = self.token_embeddings(input_ids)
#   x = recursive_forward(list(self.layers), x)
#   x = self.ln_final(x)
#   return self.lm_head(x)
```

(b) Best single-level checkpointing for xl model. With a single level of recomputation (no nested checkpoints), we partition the N=32 Transformer blocks into contiguous groups of G blocks and wrap each group in `torch.utils.checkpoint`. During backward, one group at a time is recomputed, so peak activation memory is

$$\underbrace{\frac{N}{G} \cdot i}_{\text{saved group inputs}} + \underbrace{G \cdot R}_{\text{one group's residuals}},$$

where i is the size of one residual-stream tensor ($4 \times 2048 \times 2560 \times 4\,\text{B} = 80\,\text{MiB}$) and R is the saved-for-backward memory per block. At context length 2048, R is dominated by the quadratic attention weight matrix ($4 \times 32 \times 2048^2 \times 4 \approx 2\,\text{GiB}$), totalling roughly 6–7 GiB per block. Because $R \gg i$, the $G \cdot R$ term dominates and the minimum is achieved at $G = 1$ (checkpoint every individual block).

Measured peak memory (xl, batch 4, ctx 2048, FP32):

Group size G	Peak memory (GiB)
1	**63.60**
2	69.56
4	81.48
8	OOM
16	OOM
32 (no ckpt)	OOM

Checkpointing every block (G=1) uses 63.60 GiB. Doubling to G=2 adds ∼6 GiB (one extra block's residuals at ctx 2048), confirming the per-block residual cost. G=4 is barely viable at 81.48 GiB, and $G \geq 8$ exceeds the 80 GiB A100 capacity. The result validates the theoretical optimum: when saved-for-backward memory per block ($R \approx 6$–7 GiB) vastly exceeds the input tensor size (i=80 MiB), the cost function is dominated by $G \cdot R$ and G=1 is the clear winner.

3 GPU Kernels

3.1 PyTorch Attention Benchmarking

Problem (pytorch_attention)

(a) Attention benchmarking at different scales. We benchmark `scaled_dot_product_attention` with batch size 8, no multi-head dimension, a causal mask, and 100 timed iterations after 5 warm-up steps. All 20 configurations run to completion on our 80 GB A100; on a smaller GPU the `seq=16384` rows would OOM. The "BW" column reports pure backward time (total loop minus forward).

d	Seq len	FW (ms)	BW (ms)	Mem before BW (MiB)	Peak (MiB)
16	256	0.090	0.339	20.8	29.0
16	1024	0.195	0.486	83.4	211.9
16	4096	5.324	11.912	1064.7	3114.7
16	8192	20.834	46.922	4193.1	12389.1
16	16384	82.968	187.140	16689.9	49465.9
32	256	0.087	0.244	21.3	29.6
32	1024	0.197	0.462	85.4	214.4
32	4096	5.314	11.902	1072.7	3124.7
32	8192	20.824	46.984	4209.1	12409.1
32	16384	83.062	187.392	16721.9	49505.9
64	256	0.088	0.251	22.3	30.8
64	1024	0.214	0.467	89.4	219.4
64	4096	5.477	12.066	1088.7	3144.7
64	8192	21.163	47.288	4241.1	12449.1
64	16384	84.913	189.233	16785.9	49585.9
128	256	0.088	0.249	24.3	33.3
128	1024	0.251	0.559	97.4	229.4
128	4096	5.725	12.624	1120.7	3184.7
128	8192	22.519	49.840	4305.1	12529.1
128	16384	89.116	196.593	16913.9	49745.9

Memory accounting. Consider d=128, seq=16384 (the largest configuration). The tensors saved for the backward pass are:

Tensor	Shape	Size (MiB)
Q, K, V (3 tensors)	$8 \times 16384 \times 128$	$3 \times 64 = 192$
Attention scores	$8 \times 16384 \times 16384$	8192
Attention weights (softmax output)	$8 \times 16384 \times 16384$	8192
Causal mask (bool)	16384×16384	256
Output	$8 \times 16384 \times 128$	64
Total (predicted)		**16,896**
Measured		**16,914**

The two seq $\times$ seq matrices (attention scores and weights) account for 16,384 out of 16,914 MiB (**96.9**%) of saved memory. Memory before backward quadruples each time the sequence length doubles, confirming $O(\text{seq}^2)$ scaling:

Seq len	Mem before BW (MiB)	Ratio from prev
256	24.3	—
1024	97.4	4.0×
4096	1120.7	11.5× (16× expected)
8192	4305.1	3.8×
16384	16913.9	3.9×

Both forward and backward time also scale quadratically with sequence length, driven by the $O(B \cdot S^2 \cdot d)$ matrix multiplications for scores and weighted values. Changing d from 16 to 128 has minimal effect on memory (since the seq^2 term dominates) but increases compute by $\sim 8\times$ at small seq where the $O(d)$ work dominates.

Eliminating the quadratic memory cost. The $O(\text{seq}^2)$ memory comes from materialising the full attention score matrix. **FlashAttention** (Dao et al., 2022) computes attention in fused tiled SRAM passes, never writing the full $S \times S$ matrix to HBM. This reduces saved-for-backward memory from $O(S^2)$ to $O(S)$ (only the softmax log-sum-exp statistics are kept), enabling much longer sequences within the same GPU memory budget. In PyTorch, `torch.nn.functional.scaled_dot_product_attenti` automatically dispatches to a memory-efficient kernel when available.

3.2 Torch Compile

Problem (torch_compile)

(a) Compiled vs. uncompiled attention. We wrap `scaled_dot_product_attention` with `torch.compile` and rerun the same benchmark grid. The table below compares forward and backward times (ms) and the speedup from compilation.

		Forward (ms)			Backward (ms)		
d	Seq	Eager	Compiled	Speedup	Eager	Compiled	Speedup
16	256	0.090	0.066	1.4×	0.339	0.123	2.8×
16	1024	0.195	0.101	1.9×	0.486	0.183	2.7×
16	4096	5.324	1.633	3.3×	11.912	4.540	2.6×
16	8192	20.834	6.891	3.0×	46.922	18.037	2.6×
16	16384	82.968	24.800	3.3×	187.140	71.615	2.6×
32	256	0.087	0.082	1.1×	0.244	0.178	1.4×
32	1024	0.197	0.107	1.8×	0.462	0.217	2.1×
32	4096	5.314	1.992	2.7×	11.902	4.702	2.5×
32	8192	20.824	9.724	2.1×	46.984	19.513	2.4×
32	16384	83.062	24.912	3.3×	187.392	71.857	2.6×
64	256	0.088	0.089	1.0×	0.251	0.176	1.4×
64	1024	0.214	0.184	1.2×	0.467	0.236	2.0×
64	4096	5.477	1.675	3.3×	12.066	4.615	2.6×
64	8192	21.163	6.314	3.4×	47.288	18.156	2.6×
64	16384	84.913	25.145	3.4×	189.233	72.147	2.6×
128	256	0.088	0.085	1.0×	0.249	0.175	1.4×
128	1024	0.251	0.208	1.2×	0.559	0.314	1.8×
128	4096	5.725	1.718	3.3×	12.624	4.733	2.7×
128	8192	22.519	6.388	3.5×	49.840	18.508	2.7×
128	16384	89.116	25.531	3.5×	196.593	73.598	2.7×

`torch.compile` provides **3–3.5× forward speedup** and **2.5–2.7× backward speedup** at large sequence lengths, by fusing element-wise operations (masking, scaling, softmax) into a single GPU kernel and eliminating intermediate memory round-trips. At small sizes (seq≤256), kernel launch overhead dominates and compile provides little benefit. Peak memory also drops significantly (e.g. d=128, seq=16384: 49.7 GiB eager → 33.4 GiB compiled, a 33% reduction) because fused backward kernels avoid materialising some intermediate gradient tensors.

(b) Compiled full Transformer model. We wrap the entire `BasicsTransformerLM` with `torch.compile(model)` and rerun the end-to-end benchmark (batch=4, ctx=512, FP32) across all model sizes. NVTX annotations are disabled for both eager and compiled runs to ensure a fair comparison (NVTX context managers cause graph breaks in Dynamo). All timings are median over 10 iterations (seconds).

	Forward (s)			Backward (s)			Optimizer (s)			Total (s)		
Size	Eager	Comp.	Spdup	Eager	Comp.	Spdup	Eager	Comp.	Spdup	Eager	Comp.	Spdup
Small (128M)	0.0105	0.0074	1.42×	0.0261	0.0182	1.43×	0.0084	0.0085	0.99×	0.0450	0.0341	1.32×
Medium (423M)	0.0263	0.0191	1.38×	0.0697	0.0473	1.47×	0.0234	0.0232	1.01×	0.1194	0.0897	1.33×
Large (969M)	0.0549	0.0395	1.39×	0.1514	0.0997	1.52×	0.0481	0.0482	1.00×	0.2543	0.1873	1.36×
XL (3.4B)	0.1428	0.1049	1.36×	0.3535	0.2569	1.38×	0.2774	0.2770	1.00×	0.7739	0.6388	1.21×

Peak memory comparison:

Size	Eager (GiB)	Compiled (GiB)	Reduction
Small	5.04	4.32	14.3%
Medium	13.72	11.84	13.7%
Large	27.49	23.95	12.9%
XL	65.54	59.20	9.7%

Key findings:

Forward pass: `torch.compile` provides a consistent **1.36–1.42× forward speedup** across all model sizes. Dynamo fuses element-wise operations (RMSNorm, SwiGLU activation, softmax scaling, causal masking) into optimised Triton kernels, reducing GPU kernel launch overhead and memory bandwidth.

Backward pass: The backward pass benefits even more, with **1.38–1.52× speedup**. Compiled backward kernels avoid materialising intermediate gradient tensors, reducing both memory traffic and the number of kernel launches.

Optimizer step: The optimizer is **unchanged** ($\approx 1.0\times$) because our custom `AdamW` class is not compiled (only the model is wrapped with `torch.compile`).

Total step: The end-to-end speedup ranges from **1.21× (XL) to 1.36× (Large)**. The XL model shows smaller total speedup because the optimizer step (277 ms, uncompiled) accounts for 36% of total step time, diluting the FW+BW gains. For FW+BW alone, the speedup is 1.37–1.48×.

Memory: Peak memory is reduced by **10–14%** because fused kernels eliminate intermediate activation tensors that would otherwise be materialised between separate eager-mode kernels.

3.3 FlashAttention-2 Benchmarking

Problem (flash_benchmarking)

(a) Triton FlashAttention-2 vs. PyTorch attention.

4 Distributed Data Parallel Training

4.1 Distributed Communication

Problem (distributed_communication_single_node)

4.2 Naïve DDP Benchmarking

Problem (naive_ddp_benchmarking)

4.3 Minimal DDP with Flat Gradients

Problem (minimal_ddp_flat_benchmarking)

4.4 DDP with Overlapping Individual Parameters

Problem (ddp_overlap_individual_parameters_benchmarking)

(a) Benchmarking overlapped DDP.

(b) Nsight profiler comparison.

5 Optimizer State Sharding

Problem (optimizer_state_sharding_accounting)

(a) Peak memory usage with and without sharding.

(b) Training speed with and without sharding.

(c) Comparison with ZeRO Stage 1.

6 Fully-Sharded Data Parallel

Problem (fsdp_accounting)

(a) Expected memory savings from FSDP.

(b) All-gather timing analysis.

7 Analyzing Parallelism Strategies

7.1 Communication Primitives

Problem (alternate_ring_all_reduce)

7.2 Data Parallel Calculations

Problem (data_parallel_calcs)

(a) FLOPs for backward pass with N_{DP} data parallelism.

(b) Communication time for backward pass.

(c) Maximum N_{DP} before communication bottleneck.

7.3 FSDP Calculations

Problem (fsdp_calcs)

(a) FLOPs for forward and backward pass with FSDP.

(b) Communication time for forward and backward pass.

(c) Maximum N_{FSDP} before communication bottleneck.

7.4 Tensor Parallel Calculations

Problem (tp_calcs)

(a) Backward pass equations for TP.

(b) FLOPs for forward and backward pass with TP.

(c) Communication time for forward and backward pass.

(d) Maximum $N_{\mathbf{TP}}$ before communication bottleneck.

7.5 2D Parallelism (FSDP + TP)

Problem (fsdp_tp_calcs)

(a) FLOPs for forward pass with 2D parallelism.

(b) Communication time for forward pass.

(c) Maximum N with overlapped communication.

(d) Maximum N without overlapped communication.

8 Leaderboard

Problem (leaderboard)